Hybrid-NL2SVA

Part 1: Improving the Hybrid-NL2SVA pipeline

Part 2: An end-to-end Spec2SVA framework

Hybrid-NL2SVA Project

New York University

September 3, 2026

Hybrid-NL2SVA: Integrating RAG and Finetuning for LLM-based NL2SVA, MLCAD 2025.

NYU

Outline

Part I

The Hybrid-NL2SVA Pipeline

Why NL2SVA is hard

Hardware verification can take up to 70% of chip design time

SVAs are small checkers built into the design: they watch the hardware run and flag it the moment real behavior breaks a rule from the spec. Writing them by hand has two steps: (1) find the rules to check, in plain English, and (2) turn each rule into an SVA – step (2) is **NL2SVA**, and it is what this project is about.

Why a general AI model struggles.

- Not much SVA code in the data these models learned from
- SVA has timing words that look almost the same but mean different things:
  - a `|->` b: b same cycle as a. a `|=>` b: b *1 cycle later*
  - a `##2` b: b *exactly* 2 cycles later. a `##[1:3]` b: b *anywhere* in cycles 1–3
- Input descriptions can be written at very different abstraction levels of detail, for the same rule:
  - *high-level idea: "the handshake should work correctly"*
  - *some detail: "that ack correctly follows a request"*
  - *fully precise: "whenever req rises, ack must be asserted exactly 1 clock cycle later"*

  the first two need grounding first (Step 1, next slide)

Our approach: three steps

Never trust one AI answer

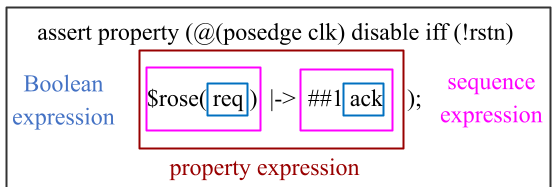
Step 1: rewrite the input into **OL-NL** (operator-level natural language) – name only real signals (e.g. `req`, `ack`, `clk`, `rstn`), and describe how they relate in plain words that match one specific operator, e.g. *"whenever req rises, ack must be asserted exactly 1 clock cycle later"* matches `$rose(req) |-> ##1 ack`, operator for operator, in plain English

Step 2: HybridRetrieval-Augmented Generation – give the model the right operator knowledge, then write a first answer

Step 3: use JasperGold to fix syntax errors; also parse the generated SVA into a syntax tree to automatically catch and fix mistakes in what it actually checks

The structure of an SVA

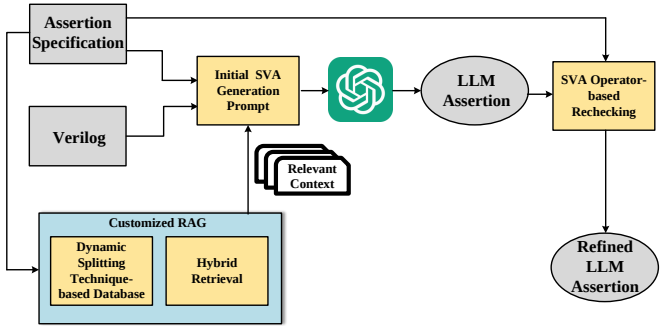
Four nested layers, each with its own operators – the source of most confusion



A Boolean expression (`req`) is wrapped by a sequence expression (`$rose(req) |-> ##1 ack`), wrapped by a property expression, wrapped by the assertion statement itself – getting any one layer's operator wrong (e.g. a sequence operator used where a property operator belongs) silently changes the property's meaning rather than raising a syntax error.

The original pipeline (MLCAD'25 paper)

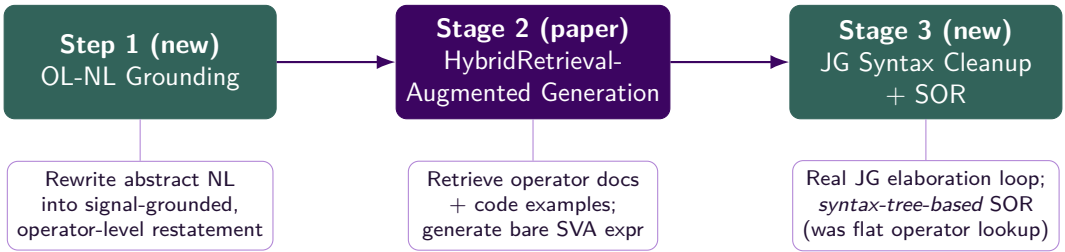
The customized RAG framework, from the MLCAD'25 paper



Customized RAG bundles two techniques: **dynamic splitting** (builds a code-centric retrieval database from SVA textbooks, preserving each code snippet with its surrounding explanation) and **HybridRetrieval** (next slide). Retrieved context enriches the Initial SVA Generation Prompt; the output is then passed through SVA Operator-based Rechecking (Stage 3) for a refined final assertion.

The pipeline today: three stages

Step 1 and Stage 3's SOR are both new – only Stage 2 is unchanged from the paper



Step 1 is only needed when the input isn't already signal-grounded. Stage 3's SOR now parses the candidate SVA into a syntax tree and rebuilds its meaning bottom-up, replacing the paper's flat per-operator lookup.

Stage 1 in detail: what is OL-NL?

Only needed when the input doesn't already name real signals

OL-NL = **operator-level natural language**: a plain-English sentence with two rules – (1) name *only* real signals from the design, never a made-up name, and (2) say exactly how those signals relate (e.g. "at the same time" or "N cycles later"); each such relation matches one specific SVA operator, but stays plain words, not code.

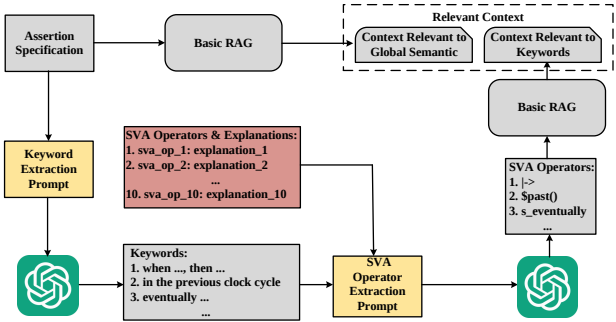
Worked example (same signals as the earlier SVA):

```
Plain description: that ack correctly follows a request.
  Real signals: req, ack, clk, rstn
OL-NL: Whenever req rises, then exactly 1 clock cycle
later, ack must be asserted.
```

Names only real signals, states the exact timing – matches `$rose(req) |-> ##1 ack` operator for operator, in plain English.

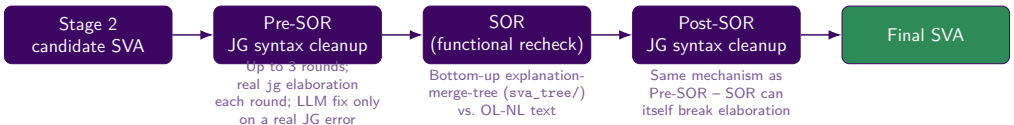
Stage 2 in detail: HybridRetrieval-augmented generation

Two independent retrieval paths feed one generation call – from the MLCAD'25 paper



Path 1 (top): Basic RAG on the full assertion specification – global-semantic similarity against the code database. **Path 2 (bottom):** an LLM decomposes the description into keyword phrases (*Keyword Extraction Prompt*), maps each to the closest of 10 SVA operators (*SVA Operator Extraction Prompt*), then a second Basic RAG pass retrieves chunks specifically discussing those operators. Both paths' **Relevant Context** feeds the single generation call.

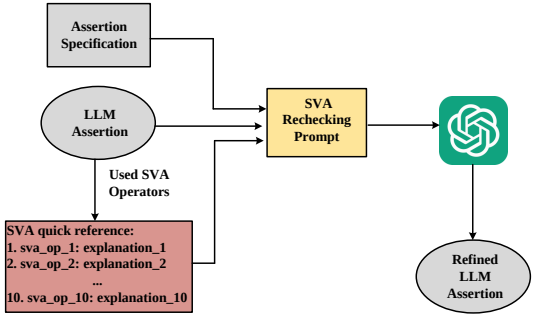
Stage 3 in detail: syntax cleanup + SOR



Confirmed live (Human-69): SOR combined two implications with an invalid `else` inside a property – without the post-SOR pass, this went straight through as the final answer. Running cleanup *twice* (once before, once after SOR) closes that gap.

As published: SVA operator-based rechecking

From the MLCAD'25 paper – the originally published SOR mechanism

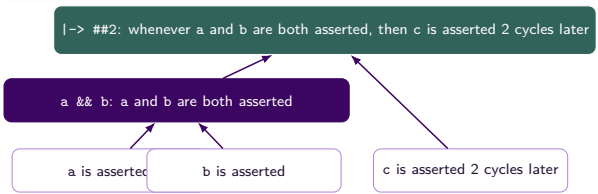


Extract the operators the LLM assertion actually used, look each one up in the SVA quick-reference table, and hand the assertion, the specification, and those explanations to a *SVA Rechecking Prompt*. **Since published:** the explanation-merge-tree (next slide) replaced this flat operator lookup with a full bottom-up rebuild of the assertion's meaning, catching mismatches the original per-operator check could miss.

Inside SOR: the explanation-merge-tree

Parse the candidate's own operator structure bottom-up, then compare

```
Candidate SVA:  a && b |-> ##2 c
```



Compare the root's composed meaning against the Stage 1 OL-NL description. Match → confirm verbatim. Mismatch → the model revises, but is told to change *only* the flagged node (e.g. just ##2), not rewrite the whole property.

The bug this talk's improvement targets

A confirmed, repeated timing error in `|=>` + `##N` together

Using `|=>` (which itself already moves forward one clock cycle) together with a written-out `##N` delay silently lands on `N+1` total cycles instead of `N` – both at **generation time** and inside **SOR's own revision step**, so the error can survive the self-correction pass meant to catch it.

Fix: avoid it at the root – always use `|->`, and always write out any delay with `##N` using the *total* cycle count, instead of combining a hidden-cycle operator with a written-out one.

SOR's explanation-merge-tree can also use fixed, rule-based templates for each operator's meaning, instead of asking an LLM to compose it – removes another source of the same timing slip.

The improvement: SOR-timing fixes vs. the original pipeline

Same benchmark rows, same pipeline, only the timing-fix flags differ

Dataset	Configuration	FM-strict	FM-relaxed
nl2sva_human_ verified (73)	Original pipeline	71.2–74.0%	83.6–87.7%
	+ SOR-timing fixes	**74.0%**	**87.7%**
nl2sva_machine_ verified (283)	Original pipeline	82.0% (232/283)	90.5% (256/283)
	+ SOR-timing fixes	**85.2% (241/283)**	**95.4% (270/283)**

A separate, unrelated bug was also fixed along the way: range/unbounded delays (`##[M:N]`, `##[*]`, `##[+]`) were losing their real bounds internally.

Benchmark & metrics

FVEval-Verified (wyt2000/FVEval-Verified): a human-expert-corrected version of FVEval's `nl2sva_human` and `nl2sva_machine` benchmarks.

- **nl2sva_human_verified** (73 rows) – expert-written ground-truth SVAs and NL descriptions
- **nl2sva_machine_verified** (283 rows) – rule-generated SVAs, LLM-written NL descriptions; bare dummy testbenches, no reset signal

Metrics (all via a real JasperGold `prop_eq_checker`):

- **SC** (Syntax Correctness) – does the candidate elaborate against the real testbench on its own?
- **FM-strict** – formally proven full equivalence to the golden SVA
- **FM-relaxed** – FM-strict plus one-directional `implies` matches

Cross-model comparison — RAG+SOR pipeline

Each row uses that dataset's best-known flag configuration

Model	Dataset	SC	FM-strict	FM-relaxed
gpt-4o	human	98.6% (72/73)	74.0% (54/73)	87.7% (64/73)
gpt-4o	machine	99.6% (282/283)	84.8% (240/283)	94.7% (268/283)
DeepSeek-V4-Flash (thinking)	human	100.0% (73/73)	**86.3%** (63/73)	**94.5%** (69/73)
DeepSeek-V4-Flash (thinking)	machine	100.0% (283/283)	**92.9%** (263/283)	**98.6%** (279/283)
gpt-5	human	100.0% (73/73)	82.2% (60/73)	91.8% (67/73)
gpt-5	machine	**interrupted (45/283)**	–	–
deepseek-chat (thinking off)	human	100.0% (73/73)	74.0% (54/73)	80.8% (59/73)
deepseek-chat (thinking off)	machine	98.9% (280/283)	85.5% (242/283)	94.3% (267/283)
qwen3-8b (Aliyun)	human	91.8% (67/73)	35.6% (26/73)	53.4% (39/73)
qwen3-8b (Aliyun)	machine	88.7% (251/283)	60.8% (172/283)	73.9% (209/283)
qwen3-14b (Aliyun)	human	91.8% (67/73)	65.8% (48/73)	79.5% (58/73)
qwen3-14b (Aliyun)	machine	95.1% (269/283)	81.6% (231/283)	90.8% (257/283)

DeepSeek-V4-Flash (thinking) was originally run and labeled "DeepSeek-R1" – DeepSeek has since silently remapped the `deepseek-reasoner` API alias to route to V4-Flash with thinking mode on; relabeled accordingly.

Cross-model comparison — raw model output (no pipeline)

Same JasperGold scorer, no RAG / OL-NL grounding / SOR

Model	Dataset	SC	FM-strict	FM-relaxed
gpt-4o (0-shot)	human	94.5% (69/73)	63.0% (46/73)	76.7% (56/73)
CodeV-SVA-8B	human	97.3% (71/73)	75.3% (55/73)	83.6% (61/73)
CodeV-SVA-14B	human	93.2% (68/73)	74.0% (54/73)	80.8% (59/73)
CodeV-SVA-8B	machine	96.5% (273/283)	83.0% (235/283)	93.3% (264/283)
CodeV-SVA-14B	machine	97.2% (275/283)	80.9% (229/283)	93.3% (264/283)
DeepSeek-V4-Flash (thinking, 0-shot)	human	98.6% (72/73)	76.7% (56/73)	93.2% (68/73)
DeepSeek-V4-Flash (thinking, 0-shot)	machine	100.0% (283/283)	90.8% (257/283)	98.2% (278/283)
deepseek-chat (thinking off, 0-shot)	human	94.5% (69/73)	54.8% (40/73)	76.7% (56/73)
deepseek-chat (thinking off, 0-shot)	machine	97.5% (276/283)	78.4% (222/283)	89.4% (253/283)
qwen3-8b (Aliyun, base)	human	89.0% (65/73)	35.6% (26/73)	57.5% (42/73)
qwen3-8b (Aliyun, base)	machine	87.6% (248/283)	51.2% (145/283)	66.8% (189/283)
qwen3-14b (Aliyun, base)	human	93.2% (68/73)	49.3% (36/73)	69.9% (51/73)
qwen3-14b (Aliyun, base)	machine	91.9% (260/283)	65.7% (186/283)	76.3% (216/283)

Takeaway: the pipeline (RAG + OL-NL grounding + SOR) lifts gpt-4o's FM-strict from 63.0%→74.0% (human) and (with the machine config) from a similarly weak 0-shot baseline to 84.8% – a real, structural gain from retrieval + self-correction, not just a stronger base model.

Case study: when the pipeline does *not* help

qwen3-8b on nl2sva_human_verified

Net effect: no real change.

- FM-strict identical: 26/73 both ways
- FM-relaxed *slightly worse*: 53.4% vs. 57.5%
- 12/73 rows flip fail→pass …
- …but a near-equal 12/73 flip pass→fail

Three ways it got worse:

- An extra, unneeded `##N` added where none was needed
- A boolean's polarity flipped while "fixing" something else
- A correct plain comparison rewritten into a non-equivalent `$onehot0({...})` form

Conclusion: the pipeline's net benefit scales with the base model's own ability to self-correct *without introducing new errors* during revision – not something that is true for every model, no matter what. (nl2sva_machine_verified does *not* show this no-change pattern for the same model: 60.8% vs. 51.2% FM-strict.)

Part II

End-to-End Spec2SVA

From ready-made benchmarks to real designs

What Part I evaluated:
NL2SVA on FVEval-Verified – properties and RTL are already picked out and ready to use, one property per row, always paired with a golden (known-correct) SVA.

What real verification looks like:
a *specification document* (PDF, dozens of pages) and a *multi-file RTL design* – no pre-extracted property list, no golden SVA to check against.

Goal: close the loop – *Spec2NL* (turn a spec + RTL into a list of natural-language verification properties) feeding directly into *our* NL2SVA pipeline, with JasperGold verifying the result end to end.

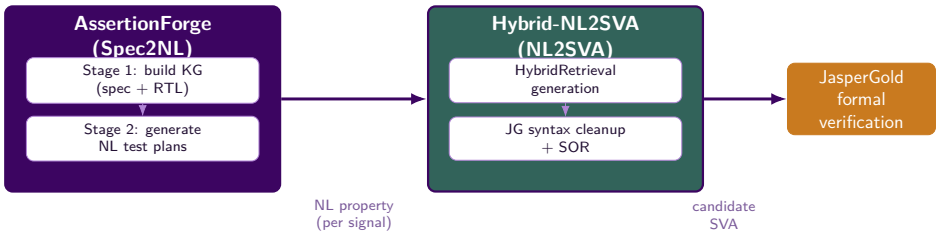
AssertionForge (NVlabs, LAD 2025)

github.com/NVlabs/AssertionForge — arXiv:2503.19174

- **Stage 1 – Knowledge Graph Construction:** builds a joint KG from the spec (via GraphRAG, domain-customized entity-extraction prompt) and the RTL (`pyverilog`-based structural analysis), then fuses the two
- **Stage 2 – Test Plan Generation:** for each architectural signal, retrieves KG-guided context (RAG chunks + graph-path analysis), prunes with an LLM judge, and generates natural-language verification *plans*

Stage 1 + Stage 2 together = **Spec2NL**: spec PDF + RTL directory → a list of natural-language properties, each grounded in a specific signal and its role in the design.

Spec2NL + NL2SVA = end-to-end Spec2SVA



NL2SVA runs with **the same settings as the nl2sva_machine_verified evaluation**, minus Step 1 (AssertionForge's plans are already signal-grounded) – plus support for a real design's actual clock name (rarely just `clk`).

Case study: UART

AssertLLM's spec + RTL

Design.

- 6 Verilog modules: `uart2bus_top` (real top), `uart_top`, `baud_gen`, `uart_rx`, `uart_tx`, `uart_parser`
- 246 total signals; 18 architectural (interface) signals across the hierarchy
- Real clock port name: `clock` (not `clk`)

Spec2NL output.

- Knowledge graph: 1,371 nodes, 1,620 edges (spec + RTL fused)
- **323** natural-language verification properties generated (all 18 signals)
- QiMeng-CodeV-SVA's own Table 5 reports **265** SVAs for UART (GPT-4o Spec2NL + GPT-4o NL2SVA) – same order of magnitude

A real gap found & fixed along the way: upstream's own signal list example only listed 2 of the 18 real signals, and its LLM-client code shipped as an empty "add your own client" stub – both needed fixing before Stage 1/2 ran end to end.

Sample generated NL properties

- **baud_clk:** "that after a reset condition, baud_clk will resume toggling without skipping cycles based on the previously set value of baud_freq"
- **baud_freq:** "that baud_freq maintains a constant value when the `global_clock_freq` and `baud_rate` inputs remain unchanged over 20 clock cycles"
- **int_req / int_gnt:** bus request/grant handshake properties derived from path analysis between `uart2bus_top`'s internal bus interface signals

Example NL2SVA output (`clock_signal=clock`, no `disable iff`, matching the machine-dataset config):

```
asrt: assert property (@(posedge clock)
    (new_tx_data && !tx_busy) |-> tx_data
);
```

UART: full NL2SVA + formal verification results

323-property run via Hybrid-NL2SVA (gpt-4o, machine config), scored with JasperGold

Spec2NL / NL2SVA	#SVA	#SynC	#Proven
QiMeng-CodeV-SVA: GPT-4o / GPT-4o	265	186 (70.2%)	54 (20.4%)
Ours – all 18 signals	323	225 (69.7%)	60 (18.6%)

Caveat: we independently reconstructed the 18-signal list ourselves (the cloned config only had a 2-signal placeholder); we have no evidence the paper's own run used the same signal scope, so this is not a strict apples-to-apples comparison – only evidence our pipeline lands in a reasonable, self-consistent range. #Proven checked with a real formal proof, no golden SVA needed.

A methodology gap: properties about free inputs

AssertionForge's own worked examples are entirely PWDATA-shaped

"that the input data PWDATA has a value between 83 and 165 ... 3 clock cycles after reset deasserted" – `PWDATA` is APB's free bus-input signal; every one of AssertionForge's 5 hardcoded style examples follows this same shape.

UART signal subset	**n**	**#Proven**
Free inputs / out-of-scope (`ser_in`, `int_rd_data`, `int_gnt`, `ce_16`)	67	11.9%
Design-controlled only (outputs + internal wires)	**256**	**20.3%**

A primary *input* port is free/unconstrained under JasperGold with no environment `assumes` – a property claiming "this input settles into range X" is nearly impossible to prove, just because of how the design is built, regardless of NL2SVA quality. Excluding these 4 signals lifts #Proven from 18.6%→**20.3%**, matching the GPT-4o/GPT-4o reference almost exactly. (`ce_16`: declared inside `uart_top`, one level below our assertion scope `uart2bus_top` – not even a valid identifier there.)

A second gap: signal hallucination, not just free inputs

Of 98 syntax-fail rows, "X' is not declared" errors name 68 distinct identifiers

Category (checked against the 6 real RTL files)	occurrences
Real RTL identifiers used at the *wrong scope* (e.g. `ce_16`×34, `bit_count`×26, `ce_1`×22)	21 distinct
Pure hallucinations, not found anywhere in the RTL (`local_global_clock_freq`, `logic_counter`, …)	47 distinct

But most of this traces back further, to the NL plans themselves: of 338 undeclared-identifier occurrences, **236 (69.8%)** have their "core" name *literally present in the corresponding NL plan's own text* – e.g. the `baud_freq` plan on the earlier slide reads *"… using the formula 16*baud_rate / gcd(global_clock_freq, 16*baud_rate)"*, and NL2SVA just copied `baud_rate/global_clock_freq` word for word as identifiers – except neither is a real RTL signal. This is a Spec2NL grounding gap, not (only) an NL2SVA one.

Root cause: real RTL documentation, not noise

`baud_gen.v`'s own header comment, verbatim

```
// the two registers should be calculated as follows:
//   baud_freq  = 16*baud_rate / gcd(global_clock_freq, 16*baud_rate)
//   baud_limit = (global_clock_freq / gcd(...)) - baud_freq
```

`baud_freq` is an *input* to `baud_gen` – `baud_rate`/`global_clock_freq` never appear as actual Verilog identifiers anywhere in the 6 files, only in this comment. True, directly-relevant content, not retrieval noise – the model has good reason to want to cite it.

Prompt-only fixes made this WORSE, not better, across several revisions – we saw the same pattern elsewhere too: naming the exact bad thing to avoid puts it right in front of the model, and it tends to copy it instead of avoiding it. Invalid-signal leakage rose 14%→22%→27%→30% across several tries, one after another, to ban it in words.

Fix: two-step generation + a mechanical filter

Reworking AssertionForge's Stage 2 directly

- **Step 1 (idea):** free-form property ideas, no precision rules at all – just "what's worth testing"
- **Step 2 (OL-NL grounding):** rewrites Step 1's ideas into precise, signal-grounded form – valid-signal whitelist, operator-level/sequential-vs-combinational discipline, vague-qualifier / trailing-rationale / gcd bans, the *same* SVA Operator Context table Hybrid-NL2SVA's own Stage 1 uses – literally the same grounding task Stage 1 performs, on AssertionForge's side instead
- **Mechanical filter (post-hoc, not a prompt):** drops any grounded plan still referencing a signal-shaped word that isn't on the real signal list – carries none of the copy-the-bad-example risk above, since it never goes back into a prompt

Lesson: banning a specific bad pattern *in words* tends to make it worse; moving the same rule *out of the prompt* into a mechanical check is what actually worked.

Validated: same 44 properties, ungrounded vs. grounded

`baud_clk/baud_freq` **pilot – the exact same starting ideas, matched one to one**

Same 44 properties, different generation	**n**	**#SynC**	**#Proven**
Raw ideas + gpt-4o 0-shot baseline (no pipeline)	44	30 (68.2%)	15 (34.1%)
Raw ideas + CodeV-SVA-8B (no pipeline)	44	18 (40.9%)	12 (27.3%)
Raw ideas + CodeV-SVA-14B (no pipeline)	44	21 (47.7%)	13 (29.5%)
Grounded (Step 2 + filter) + full Hybrid-NL2SVA	**44**	**40 (90.9%)**	**24 (54.5%)**

Same 44 properties both ways (matched one to one) – grounding + the full pipeline is **+22.7pp #SynC, +20.4pp #Proven** over the best ungrounded baseline; a specialized NL2SVA model alone (CodeV-SVA) can't make up for bad, made-up signal names the way grounding does. *Scope: `baud_clk/baud_freq` only (2/18 signals) – a small test before the remaining 16 signals and a full re-run.*

Next steps

- Run the two-step generation + mechanical filter on the remaining 16 signals and fully regenerate `nl_plans_uart.txt`
- Extend the pilot to the remaining 4 designs from QiMeng-CodeV-SVA's Table 5 (APB, ETHMAC, OPENMSP430, SOCKIT)
- Swap AssertionForge's own NL2SVA backend for a head-to-head comparison at the *same* Spec2NL properties
- Try DeepSeek-V4-Flash / qwen3 as AssertionForge's own Spec2NL backend, not just gpt-4o
- Look into why #SynC (69.7%) trails #SVA – categorize the elaboration failures the same way Part I's row-by-row diffing did

Thank you